GeoInspect

Architecture and Technical Design

GIS-enabled field inspection assignment with four independently runnable services, database-per-service, concurrency control, idempotency, and resilient notifications.

System	GIS-enabled field inspection platform
Repository	https://github.com/sharmaark/SS_Assignment
Document	Technical design and implementation reference
Prepared	2026-10-13

Executive overview

GeoInspect coordinates the assignment of field inspections to eligible inspectors. Its design protects scarce inspector availability, preserves one outcome for repeated requests, and makes notification failures visible without invalidating confirmed work. The system uses REST APIs and independently owned PostgreSQL and MongoDB databases.

Document guide

Section	Page
1. Domain, scope and service boundaries	2
2. Architecture and communication	3
3. Data ownership and API contracts	4
4. Concurrency and request idempotency	5
5. Resilience and verification	6
6. Design decisions and operational limits	7

Supporting technical references

File	Purpose
diagrams/01_System_Architecture.pdf + .drawio	Service boundaries, databases, protocols, and ownership
diagrams/02_Main_Assignment_Sequence.pdf + .drawio	Happy-path transaction across services
diagrams/03_Notification_Failure_Sequence.pdf + .drawio	Timeout, retry, graceful failure, and recovery
diagrams/04_Data_Ownership.pdf + .drawio	Per-service entities and integrity constraints
services/* and scripts/*	Source code, unit tests, startup, end-to-end, and challenge scripts

1. Domain, scope and service boundaries

Field coordinators assign geographically distributed inspections to inspectors. The same inspector and slot can be requested concurrently; retries can create duplicate assignments; and the notification provider may fail after the operational work has succeeded.

Scope is deliberately API-only: fixed slots, WGS84 points, rectangular service areas, synthetic data, and a mock notification provider. Authentication, UI, routing, and real SMS/email are excluded.

Business rules

Rule	Enforced behavior	Code / test
BR1	Inspector service area covers the inspection point; boundaries are included.	workforce/src/business.py + tests
BR2	Inspector has the required skill.	workforce/src/business.py + tests
BR3	One inspector reservation per exact time slot.	workforce/src/data.py UNIQUE + tests
BR4	Only the assignment owning a work-order claim may confirm it.	work_order/src/business.py + tests
BR5	Same idempotency key with changed payload returns HTTP 409.	dispatch/src/business.py + tests
BR6	Only assigned jobs accept size-limited JPEG/PNG photos.	work_order/src/business.py + tests

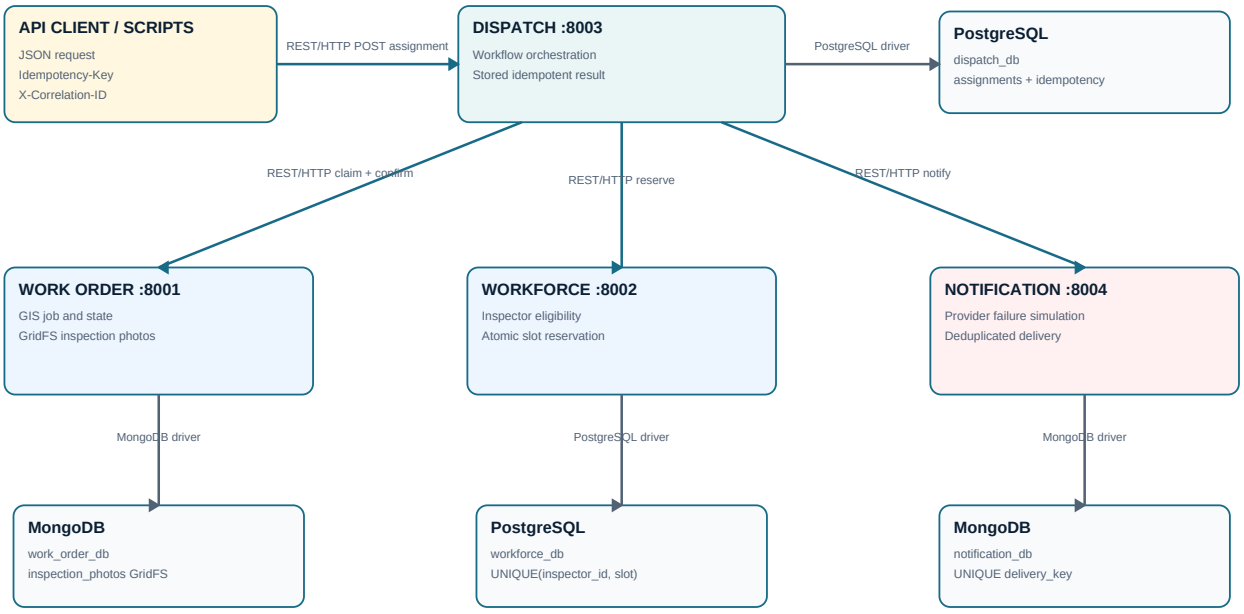
Service responsibilities

Service	One responsibility	Owned database
Work Order :8001	Inspection location, skill, state, and GridFS photos	MongoDB work_order_db
Workforce :8002	Inspector eligibility and atomic slot reservation	PostgreSQL workforce_db
Dispatch :8003	Assignment orchestration and stored idempotent results	PostgreSQL dispatch_db
Notification :8004	Failure simulation and deduplicated delivery	MongoDB notification_db

The boundaries follow business capabilities. Services exchange IDs through REST APIs and never read another service's database. A shared database and service-per-table split were rejected because both weaken ownership and create a distributed CRUD system.

2. Architecture and communication

Dispatch is the workflow orchestrator. It claims the work order, asks Workforce to reserve the inspector and slot, confirms the work order, and then requests notification. X-Correlation-ID is propagated through every HTTP call.



Each service accesses only its owned database. Cross-service references are IDs carried through REST/HTTP JSON.

Why this communication design

REST provides immediate eligibility and contention feedback. The client makes one request to Dispatch, while Dispatch calls each capability service directly; there is no long A-to-B-to-C-to-D service chain. Local database constraints protect strong invariants, while persisted workflow state makes partial failure visible.

Detailed views

Open diagrams/01_System_Architecture for the component view, diagrams/02_Main_Assignment_Sequence for the happy path, and diagrams/03_Notification_Failure_Sequence for the failure path. Each is supplied as a presentation-ready PDF and editable draw.io file.

3. Data ownership and API contracts

Data ownership

Service	Core records	Integrity rule
Work Order	work_orders; inspection_photos GridFS bucket	OPEN -> CLAIMED -> ASSIGNED; claim owner controls confirmation
Workforce	inspectors; reservations	UNIQUE(inspector_id, slot); UNIQUE(assignment_id)
Dispatch	assignments; idempotency_records	Primary key on Idempotency-Key; SHA-256 request fingerprint
Notification	notifications	Unique delivery_key keeps retry effects singular

MongoDB GridFS stores photo bytes in chunks and metadata inside Work Order ownership. PostgreSQL is used where relational uniqueness protects reservation and idempotency invariants. Foreign IDs are values only; there are no cross-service foreign keys or joins.

API summary

Service	Important versioned endpoints	Responses
Work Order	POST /api/v1/work-orders; POST /{id}/claim|confirm|release-claim; POST /{id}/photos	201, 200, 404, 409, 422
Workforce	POST /api/v1/inspectors; POST /api/v1/reservations	201, 409, 422
Dispatch	POST /api/v1/assignments; POST /{id}/retry-notification	201, 202, 409, 422, 503
Notification	POST /api/v1/notifications; PUT /api/v1/mock-mode	201, 200 replay, 202, 503

Error model

```
{"error":{"code":"SLOT_UNAVAILABLE","message":"Inspector is already reserved for this slot","details":{}},"correlation_id":"..."}
```

Every service exposes GET /health, Swagger UI at /docs, generated /openapi.json, environment-based configuration, and structured correlation-aware logs. Committed specifications are under docs/openapi/.

4. Concurrency and request idempotency

Concurrency control

Recorded result: 20 parallel assignment requests compete for capacity N=1. Exactly 1 succeeds, 19 return HTTP 409, and PostgreSQL contains one reservation.

Workforce attempts the reservation INSERT directly. PostgreSQL's UNIQUE(inspector_id, slot) constraint selects one winner across processes and replicas; integrity errors become SLOT_UNAVAILABLE. A Python check followed by INSERT would race because two requests can both read zero rows before either writes.

Evidence	Claim
scripts/challenge_1_concurrency.py	Creates 20 eligible assignments concurrently
evidence/challenge-1-concurrency/output.txt	successes=1; conflicts=19; final_reservations=1

Request idempotency

Recorded result: two simultaneous identical calls and a sequential replay return HTTP 201 with the same assignment ID; changed payload with the same key returns HTTP 409.

Dispatch stores Idempotency-Key, canonical SHA-256 fingerprint, assignment ID, status code, and response body. The unique key selects one owner; a concurrent duplicate waits for and replays the stored result. A changed fingerprint is rejected. Stable downstream assignment and delivery keys prevent duplicate reservation and notification effects.

Evidence	Claim
scripts/challenge_2_idempotency.py	Concurrent, sequential, and mismatched-payload cases
evidence/challenge-2-idempotency/output.txt	One assignment result, one reservation, one notification

5. Resilience and verification

Notification resilience

Dispatch uses a configured timeout, three total attempts, exponential backoff, and jitter. A failed provider leaves a valid assignment CONFIRMED with notification PENDING. After recovery, POST /api/v1/assignments/{id}/retry-notification changes it to SENT. In commit-then-slow mode, the provider commits before the HTTP response is lost; the same delivery key makes the retry return the existing notification instead of sending twice.

Forced condition	Observed state	Evidence
Provider fails	Assignment CONFIRMED; notification PENDING	challenge-3a-resilience/output.txt
Provider recovers	Manual retry returns notification SENT	same output
Commit then response timeout	Notification SENT; exactly one notification record	same output

Testing results

Level	Command	Recorded result
Unit	python -m pytest	18 passed
End to end	python scripts/end_to_end.py	CONFIRMED, SENT, GridFS photo uploaded/listed
Concurrency	python scripts/challenge_1_concurrency.py	1 success, 19 conflicts
Idempotency	python scripts/challenge_2_idempotency.py	Concurrent replay safe; changed request rejected
Resilience	python scripts/challenge_3a_resilience.py	Failure, recovery, and response-loss cases pass

Raw outputs are stored under evidence/challenge-1-concurrency, evidence/challenge-2-idempotency, evidence/challenge-3a-resilience, and evidence/tests. These files capture the observed behavior of the recorded test runs. Repeat the scripts after changes to transaction or persistence logic and preserve the raw outputs alongside their timestamps.

6. Design decisions and operational limits

Key decisions and trade-offs

Decision	Why selected	Limit
REST orchestration	Immediate conflict feedback and simple tracing	Dispatch coordinates partial failures
Exact fixed slots	Small, demonstrable contention key	Does not detect overlapping intervals
Database uniqueness	Correct across concurrent processes and replicas	Schema encodes the capacity model
MongoDB GridFS photos	Self-contained streaming storage with metadata	Object storage scales better
Keep assignment when notification fails	Messaging is secondary to valid operational work	Requires visible PENDING state and retry

Limitations and future work

- Add automatic outbox workers and idempotency-record retention for production.
- Add GeoJSON/2dsphere or PostGIS only when proximity search and routing enter scope.
- Add service authentication, role-based access, malware scanning, and object storage before real deployment.

Recovery boundaries

Notification recovery is explicit and initiated through the retry API. The current implementation has no background workflow reconciler. An interrupted core transaction can leave an assignment PENDING; a duplicate request waits for a stored outcome and may return HTTP 202. Automatic takeover and full crash recovery are future work.

Engineering tooling

OpenAI Codex assisted with planning, implementation scaffolding, test scripts, documentation, and review. Recorded verification results are linked in this document; generated material remains subject to engineering review.